

C++ COMPETITIVE PROGRAMMING + DATA STRUCTURES + ALGORITHMS MASTER TEXTBOOK

PART 19

Bit Manipulation

Binary representation, the core bit tricks, XOR properties, subset generation via bitmasks, and a first look at bitmask DP — a Phase-16 topic filled in out of sequence

PART 19

Bit Manipulation

A note on numbering, then the fastest operations your CPU offers

** Where this Part fits**

This Part covers Phase 16 of the Master Roadmap (Bit Manipulation), which was skipped when the earlier Parts were being delivered. It's filed here as Part 19, after Part 18 — Advanced Data Structures, rather than being inserted earlier and renumbering everything you've already downloaded. Content-wise it doesn't depend on Parts 15–18, so reading it out of strict roadmap order costs nothing.

Every value a computer stores is, underneath, a sequence of bits. Most code never needs to think at that level — but a specific, recurring family of competitive-programming problems (subset enumeration, single-unique-element puzzles, ultra-fast parity/power-of-two checks) is solved dramatically faster, and often more simply, by operating on those bits directly instead of on the number as a whole.

Contents of Part 19

- 1. Binary Representation and Two's Complement
- 2. Bitwise Operators: AND, OR, XOR, NOT, Shifts
- 3. Core Bit Tricks
- 4. XOR Tricks
- 5. Subset Generation via Bitmasks
- 6. Bitmask DP — Introduction
- Part 19 Revision, Cheat Sheet, and Interview Questions

Chapter 1 — Binary Representation and Two's Complement

1.1 Decimal to binary, binary to decimal

Every integer is stored as a sequence of bits, each bit worth a power of two. Converting decimal to binary repeatedly divides by 2, recording remainders; converting binary to decimal sums the powers of two where a bit is 1.

Decimal	Binary (8-bit)	How to read it
13	00001101	$8 + 4 + 1 = 13$ (bits at positions 3, 2, 0)
37	00100101	$32 + 4 + 1 = 37$ (bits at positions 5, 2, 0)
255	11111111	all 8 bits set = $2^8 - 1$
1	00000001	only bit 0 set

Bit positions are numbered from 0 (rightmost, least significant) upward — this indexing convention matters for every trick in this Part.

1.2 Two's complement: how negative numbers are stored

C++ signed integers use two's complement: to negate a number, flip every bit, then add 1. This single rule is what makes addition, subtraction, and comparison work identically for positive and negative numbers at the hardware level, with no special-casing.

Step	Value (8-bit example)
Start: 5	00000101
Flip every bit (one's complement)	11111010
Add 1 → this is -5	11111011

✍ Why the top bit signals negative

In two's complement, the most significant bit doubles as a sign indicator: 0 for non-negative, 1 for negative. This is a consequence of the flip-and-add-1 rule, not a separately stored flag — there is no dedicated “sign bit field” in the hardware sense, just the natural result of the encoding.

1.3 Why this matters for bit manipulation code

Pitfall	Why it happens	Fix
Right-shifting a negative int (<code>>></code>) fills with 1s, not 0s	Two's complement preserves sign under arithmetic right shift	Use unsigned types when you need a logical (zero-fill) shift
<code>-n</code> in binary is NOT the same bit pattern as <code>n</code> with the sign flipped naively	Two's complement is flip-then-add-1, not just a sign flag	Trust the language's built-in negation; don't hand-roll bit flips to negate
<code>n & (-n)</code> relies on two's complement to isolate the lowest set bit	<code>-n</code> 's bit pattern, under two's complement, cancels every bit except the lowest set one	This is actually a FEATURE (Chapter 3) once understood, not a bug

✍ Quick Revision

- Bit position 0 is the rightmost (least significant) bit; positions increase leftward.
- Two's complement negation: flip every bit, then add 1 — this is why addition/subtraction work uniformly for signed numbers.
- The top bit of a two's-complement number indicates sign as a side effect of the encoding, not a separately stored flag.
- Right-shifting a negative signed int in C++ is implementation-defined/arithmetic (sign-preserving) — use unsigned types for a guaranteed logical shift.

Chapter 2 — Bitwise Operators: AND, OR, XOR, NOT, Shifts

2.1 The truth tables

a	b	a & b (AND)	a b (OR)	a ^ b (XOR)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

In one line each: AND is 1 only if BOTH bits are 1. OR is 1 if EITHER bit is 1. XOR is 1 if the bits DIFFER. NOT (~) flips every bit of a single operand.

2.2 Worked example

a = 12 (1100), b = 10 (1010):

Expressi on	Binary computation	Decimal result
a & b	1100 & 1010 = 1000	8
a b	1100 1010 = 1110	14
a ^ b	1100 ^ 1010 = 0110	6
~a (32-bit)	flip all 32 bits of 12	-13 (two's complement)

⚠ ~a on a signed int is almost never the small number you expect

~12 is not “-(complement within 4 bits)” — it flips all 32 (or 64) bits of the actual storage, landing on -13 due to two's complement (Chapter 1). Masking with & is usually what's actually wanted when only a few low bits matter, e.g. ~a & 0xF.

2.3 Shift operators

Operator	Effect	Equivalent to
$a \ll k$	shift bits left by k , fill with 0 on the right	$a \times 2^k$
$a \gg k$	shift bits right by k (arithmetic if signed, sign-extends)	$a \div 2^k$, floor

```
int x = 5;           // 0000...0101
cout << (x << 2);    // 20 -> 0000...10100  (5 * 4)
cout << (x >> 1);    // 2  -> 0000...0010  (5 / 2, floor)

int mask = 1 << k;   // a single 1-bit at position k -- the
                    // standard building
                    // block for nearly every trick in Chapter
                    // 3
```

✓ $1 \ll k$ is the single most-used expression in this whole Part

It produces a number that is 0 everywhere except bit position k , which is 1 — exactly the “mask” needed to check, set, clear, or toggle one specific bit. Every operation in Chapter 3 builds on this one expression.

Practice — Chapter 2

Problem 2.1 — Predict the output [Level 1]

What does $(6 \wedge 3) \& (5 \mid 2)$ evaluate to? Work it out in binary by hand, then verify.

Hint 1: $6=110$, $3=011$, XOR gives $101=5$. $5=101$, $2=010$, OR gives $111=7$. Then $5 \& 7$.

Approach: $6 \wedge 3 = 101 = 5$. $5 \mid 2 = 111 = 7$. $5 \& 7 = 101 = 5$. Final answer: 5.

Complexity: $O(1)$.

Chapter 3 — Core Bit Tricks

3.1 The complete toolkit

Goal	Expression	Idea
Check if odd/even	$n \& 1$	bit 0 is 1 iff n is odd
Check k th bit	$(n \gg k) \& 1$	shift the target bit to position 0, mask it
Set k th bit	$n \mid (1 \ll k)$	OR with a mask that's 1 only at position k
Clear k th bit	$n \& \sim(1 \ll k)$	AND with a mask that's 0 only at position k
Toggle k th bit	$n \wedge (1 \ll k)$	XOR with a mask that's 1 only at position k
Remove lowest set bit	$n \& (n - 1)$	see 3.2 for why this works
Isolate lowest set bit	$n \& (-n)$	see 3.3 for why this works — this is the Fenwick Tree trick from Part XVIII
Check power of two	$n > 0 \&\& (n \& (n-1)) == 0$	a power of two has exactly one set bit; removing it leaves 0

3.2 Why $n \& (n-1)$ removes the lowest set bit

Subtracting 1 flips every bit from the lowest set bit downward: that bit becomes 0, and every 0 below it becomes 1. ANDing with the original n then cancels exactly that lowest set bit, since the bits below it disagree (0 vs 1) and the bits above are unchanged (still equal).

$n = 12$	binary
n	1100
$n - 1$	1011
$n \& (n-1)$	1000 — the lowest set bit (4) is gone, 8 remains

3.3 Why $n \& (-n)$ isolates the lowest set bit

$-n$, under two's complement (Chapter 1), is $\sim n + 1$. Every bit above the lowest set bit of n gets flipped by $\sim n$, then flipped back by the $+1$'s carry propagation up to that same lowest set bit — everywhere except at and below the lowest set bit, n and $-n$ end up as exact opposites, and AND cancels them to 0. At the lowest set bit itself, both n and $-n$ agree (both are 1), so it survives.

$n = 12$	binary
n	0000 1100
$-n$ (two's complement)	1111 0100
$n \& (-n)$	0000 0100 — only the lowest set bit (4) survives

 This exact expression powers the Fenwick Tree

Part XVIII's Fenwick Tree update/query loops move by $i \& (-i)$ at every step — that entire data structure is built on this one bit trick.

3.4 Counting set bits

```
int countSetBits(int n) {
    int count = 0;
    while (n) {
        n = n & (n - 1); // removes the lowest set bit each
                          iteration
        count++;
    }
    return count;        // loop runs once per SET bit, not
                          once per bit
}

// In practice, prefer the compiler builtin:
// __builtin_popcount(n)      for unsigned int
// __builtin_popcountll(n)    for unsigned long long
```


✓ Why the $n \& (n-1)$ loop beats checking every bit

Checking all 32 bits one by one always loops 32 times. The $n \& (n-1)$ version loops exactly once per SET bit — for a number like 8 (binary 1000, only one set bit), it finishes in a single iteration instead of 32.

Practice — Chapter 3

Problem 3.1 — Set, clear, and toggle a specific bit [Level 1]

Given $n = 20$ (10100), set bit 1, then clear bit 4, then toggle bit 0. Trace each step in binary.

Hint 1: Apply each expression from the table in 3.1 one at a time, writing out the binary after each step.

Approach: $20 = 10100$. Set bit1: $10100 | 00010 = 10110$ (22). Clear bit4: $10110 \& \sim 10000 = 00110$ (6). Toggle bit0: $00110 \oplus 00001 = 00111$ (7).

Complexity: $O(1)$ per operation.

Problem 3.2 — Count set bits in a range $[1, n]$ [Level 2]

Compute the sum of `countSetBits(i)` for i from 1 to n .

Hint 1: Brute force: call `countSetBits` for each i — $O(n \log(\text{max value}))$. This is usually fast enough unless n is very large.

Complexity: $O(n \log(\text{max value}))$ brute force.

Chapter 4 — XOR Tricks

4.1 The three properties that make XOR special

Property	Statement	Why it's useful
Self-cancelling	$a \oplus a = 0$	pairs of equal values vanish when XORed together
Identity	$a \oplus 0 = a$	XORing with 0 changes nothing — a safe starting accumulator
Commutative & associative	order doesn't matter: $a \oplus b \oplus c = c \oplus a \oplus b$	you can XOR a whole array in any order and get the same result

4.2 Find the single unique element

Given an array where every element appears exactly twice except one, find that one element.

```
int findUnique(vector<int>& a) {
    int result = 0;
    for (int x : a) result ^= x;    // every paired value cancels
    return result;                  // via a^a=0
    // only the unpaired value
    survives
}
```

Dry run on [4, 1, 2, 1, 2]:

Step	result after XOR
start	0
$\wedge 4$	4
$\wedge 1$	5
$\wedge 2$	7
$\wedge 1$	6 (the two 1's have now cancelled: $5 \wedge 1 \wedge \dots = \dots$ simplifies to leaving $4 \wedge 2$)

Step	result after XOR
$\wedge 2$	4 (the two 2's have also cancelled) — final answer: 4

4.3 Find the missing number in [0, n]

Given an array of n distinct numbers from the range $[0, n]$ with exactly one missing, find it — without the $O(n)$ sum-formula approach (which risks overflow on large n) and without extra space.

```
int findMissing(vector<int>& a) {
    int n = a.size();           // array has n elements, values
                                // range [0, n]
    int result = n;             // start by including the "extra"
                                // value n itself
    for (int i = 0; i < n; i++)
        result ^= i ^ a[i];     // XOR in every index AND every
                                // array value
    return result;              // every present value cancels
                                // its matching index
}
```

Why this avoids overflow

The classic approach sums $0..n$ and subtracts the array's sum, which can overflow for large n (Part I's overflow habit). XOR never grows unboundedly — it stays within the same bit width throughout, sidestepping the issue entirely.

4.4 Swap two variables without a temp variable

```
a ^= b;
b ^= a; // b becomes original a (since a is now a^b, and
b^(a^b) = a)
a ^= b; // a becomes original b
```

⚠ A classic trap: a and b must be different variables

If a and b are actually the same memory location (e.g. `swapXor(v[i], v[i])`), this sequence zeroes it out instead of “swapping” it with itself — `a ^= a` sets it to 0 on the first line. In practice, `std::swap` is simpler, equally fast, and doesn't carry this risk; this trick is shown for its problem-solving value (interview questions, XOR intuition-building), not as a recommended default.

🔑 Quick Revision

- XOR of any value with itself is 0; XOR with 0 changes nothing; XOR is commutative and associative.
- Find-the-unique-element and find-the-missing-number both exploit pairs cancelling under XOR — $O(n)$ time, $O(1)$ space, no overflow risk.
- The XOR-swap trick works but silently fails (zeroes the value) if both operands are the same variable — prefer `std::swap` in real code.

Practice — Chapter 4

Problem 4.1 — Two unique elements [Level 4]

Given an array where every element appears exactly twice except TWO elements (each appearing once), find both unique elements.

Hint 1: XOR the whole array first — the result is $x \oplus y$ (the two unique values XORed together), since all paired values cancel.

Hint 2: Find any set bit in $(x \oplus y)$ — x and y must differ at that bit position. Split the array into two groups by that bit, and XOR each group separately.

Key observation: A set bit in $x \oplus y$ proves x and y differ there — splitting the whole array by that bit guarantees x and y land in DIFFERENT groups, while every properly-paired value stays together in the SAME group as its pair (since pairs are identical and share every bit).

Complexity: $O(n)$ time, $O(1)$ space.

Chapter 5 — Subset Generation via Bitmasks

5.1 The core idea: a number as a subset

For a set of n elements, every subset can be represented as an n -bit number: bit i is 1 if element i is included, 0 if it's excluded. Looping a mask from 0 to $2^n - 1$ visits every possible subset exactly once — a compact, index-free alternative to the include/exclude backtracking recursion from Part X.

mask (binary, $n=3$)	mask (decimal)	Subset of {A, B, C}
000	0	{ } (empty set)
001	1	{A}
010	2	{B}
011	3	{A, B}
100	4	{C}
101	5	{A, C}
110	6	{B, C}
111	7	{A, B, C}

```
int n = a.size();
for (int mask = 0; mask < (1 << n); mask++) {
    vector<int> subset;
    for (int i = 0; i < n; i++)
        if (mask & (1 << i))           // is element i included in
            subset.push_back(a[i]);     this subset?
    // process `subset` here
}
```

✓ Backtracking (Part X) vs. bitmask enumeration — same output, different shape

Part X's subsets() function generates the same 2^n subsets via recursive include/exclude choices. The bitmask version is iterative, often shorter, and makes it trivial to iterate subsets in numeric order or jump to a specific subset by index — but both are $O(2^n)$, and both are worth knowing.

5.2 Iterating over the bits actually set in a mask

Instead of checking every bit position, jump directly from one set bit to the next — useful when a mask is sparse.

```
int mask = 0b10110;           // bits 1, 2, 4 are set
for (int m = mask; m; m &= (m - 1)) {
    int lowestBit = m & (-m);   // isolate it (Chapter 3.3)
    int pos = __builtin_ctz(lowestBit); // count trailing zeros
    // = bit position
    cout << pos << " ";       // prints: 1 2 4
}
```

5.3 Iterating over all submasks of a given mask

A classic, slightly advanced pattern: for a fixed mask, visit every submask (every subset of its set bits) — used in some bitmask-DP transitions (Chapter 6).

```
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
    // process `sub`, a submask of `mask`
}
// note: this loop does NOT visit the empty submask (0) -- add
// it separately
// if needed
```

✍ Why (sub - 1) & mask stays a submask

sub - 1 removes the lowest set bit and sets everything below it, exactly like Chapter 3.2 — ANDing with mask then clips that result back down to only the bits mask actually allows, guaranteeing the result is still a subset of mask's bits.

Practice — Chapter 5

Problem 5.1 — Sum of all subset sums [Level 2]

Given an array of n elements, compute the sum of the sums of all 2^n subsets.

Hint 1: Each element appears in exactly half of all subsets ($2^{(n-1)}$ of them) — you don't need to enumerate subsets at all.

Key observation: Recognizing that brute-force bitmask enumeration ($O(n * 2^n)$) can sometimes be replaced by a closed-form $O(n)$ observation is itself a valuable skill — always ask whether enumeration is actually necessary.

Complexity: $O(n)$ with the closed-form insight; $O(n \times 2^n)$ via brute-force enumeration.

Problem 5.2 — Maximum AND value pair via submask enumeration [Level 4]

Given constraints suited to bitmask DP (small n , or small value range), explore how submask enumeration reduces a seemingly $O(3^n)$ -looking search.

Hint 1: The classic fact: iterating all (mask, submask) pairs across all masks costs $O(3^n)$ total, not $O(4^n)$ — because each element independently is either not in mask, in mask but not submask, or in both, giving 3 states per element.

Key observation: This $O(3^n)$ bound is a standard, frequently-cited fact worth memorizing — it's what makes submask-enumeration DP transitions tractable for n up to about 15-20.

Complexity: $O(3^n)$ total across all mask/submask pairs.

Chapter 6 — Bitmask DP: Introduction

6.1 When a DP state needs to be a SET, not a number

Some DP problems need to track “which subset of items has been used/visited/assigned so far” as part of the state — not just a single index. Representing that subset as a bitmask, and using it as an array index, is what “bitmask DP” means. This is only practical when the set is small (roughly $n \leq 20$), since the state space is $O(2^n)$.

Clue in the problem	Why bitmask DP fits
"Visit every city exactly once" (Traveling Salesman)	State needs to remember WHICH cities have been visited, not just how many
"Assign n tasks to n workers"	State needs to remember which tasks are already assigned
Small n (typically $n \leq 15-20$)	2^n states must be affordable in both time and memory

6.2 Worked example: Traveling Salesman (minimum cost)

$dp[mask][i]$ = minimum cost to have visited exactly the set of cities in mask, ending at city i. This is the canonical bitmask DP problem.

```

// dp[mask][i]: min cost, visited = mask, currently at city i
// dist[i][j]: cost to travel from city i to city j
int n;
vector<vector<int>> dist;
vector<vector<int>> dp;    // size (1<<n) x n, init to INF

int solveTSP() {
    dp[1][0] = 0;          // start at city
    0, only city 0 visited
    for (int mask = 1; mask < (1 << n); mask++) {
        for (int i = 0; i < n; i++) {
            if (!(mask & (1 << i))) continue; // i must be in
            mask
            if (dp[mask][i] == INT_MAX) continue;
            for (int j = 0; j < n; j++) {
                if (mask & (1 << j)) continue; // j must NOT be
                visited yet
                int newMask = mask | (1 << j);
                dp[newMask][j] = min(dp[newMask][j], dp[mask][i]
                + dist[i][j]);
            }
        }
    }
    int best = INT_MAX;
    int fullMask = (1 << n) - 1;
    for (int i = 0; i < n; i++)
        best = min(best, dp[fullMask][i] + dist[i][0]); //
        return to start
    return best;
}

```

6.3 Reading the state-transition table

Piece	Meaning
mask	which cities have already been visited
i	which city the path currently ends at
dp[mask][i]	minimum cost to reach this exact (visited-set, current-city) combination
mask (1 << j)	the new visited-set after also visiting city j

Piece	Meaning
Final answer	best over all i of $dp[fullMask][i] + \text{cost to return to the start}$

6.4 Complexity

Quantity	Value
Number of states (mask, i)	$O(2^n \times n)$
Work per state (loop over j)	$O(n)$
Total time	$O(2^n \times n^2)$
Compare to brute-force permutations	$O(n!)$ — bitmask DP is a dramatic improvement for n up to $\sim 18-20$

✓ Recognizing when a DP problem needs a bitmask state

If the transition needs to know exactly WHICH of a small number of items have been used (not just a count), and n is small enough that 2^n is affordable, a bitmask dimension in the DP state is almost certainly the intended approach. This is the natural meeting point of Part XV (Dynamic Programming) and this Part.

🔗 Quick Revision

- A bitmask represents a subset as an integer; looping mask from 0 to $2^n - 1$ enumerates every subset.
- $\text{mask} \& (\text{mask}-1)$ strips subsets down to submasks in $O(1)$ per step; $(\text{sub}-1) \& \text{mask}$ walks all submasks of a fixed mask.
- Bitmask DP adds a 'which subset has been used' dimension to a DP state — practical for roughly $n \leq 20$ since the state space is $O(2^n)$.
- The TSP bitmask DP reduces $O(n!)$ brute force to $O(2^n \times n^2)$ — the standard example every competitive programmer should be able to derive from scratch.

Practice — Chapter 6

Problem 6.1 — Count ways to partition into pairs (Perfect Matching) [Level 4]

Given $2k$ people and a compatibility matrix, count the number of ways to pair everyone up such that every pair is compatible.

Hint 1: $dp[mask]$ = number of ways to pair up the people in mask. Find the lowest-indexed person in mask not yet paired, and try pairing them with every other available person.

Key observation: Fixing WHICH person to pair next (always the lowest-indexed unpaired one) avoids counting the same pairing twice in a different order — a common bitmask-DP technique for avoiding overcounting.

Complexity: $O(2^n \times n)$ with the fixed-lowest-bit technique.

Part 19 — Master Revision

Bit Trick Quick Reference

Goal	Expression
Check odd/even	<code>n & 1</code>
Check kth bit	<code>(n >> k) & 1</code>
Set kth bit	<code>n (1 << k)</code>
Clear kth bit	<code>n & ~(1 << k)</code>
Toggle kth bit	<code>n ^ (1 << k)</code>
Remove lowest set bit	<code>n & (n - 1)</code>
Isolate lowest set bit	<code>n & (-n)</code>
Check power of two	<code>n > 0 && (n & (n-1)) == 0</code>
Count set bits	<code>__builtin_popcount(n)</code>
Find single unique (others paired)	XOR the whole array
Iterate submasks of mask	<code>for (sub=mask; sub>0; sub=(sub-1)&mask;)</code>

Must Remember

Quick Revision

- $1 \ll k$ builds a mask with only bit k set — the foundation of check/set/clear/toggle.
- $n \& (n-1)$ removes the lowest set bit; $n \& (-n)$ isolates it — both rely on two's complement.
- XOR's self-cancelling property ($a \oplus a = 0$) solves unique-element and missing-number problems in $O(n)$ time, $O(1)$ space, with no overflow risk.
- A bitmask represents a subset as an integer; enumerating 0 to $2^n - 1$ visits every subset.
- Bitmask DP adds a subset-of-used-items dimension to a DP state, practical for roughly $n \leq 20$; TSP is the canonical example, $O(2^n \cdot n^2)$ vs. $O(n!)$ brute force.

Common Mistakes Recap

Mistake	Fix
Assuming $\sim a$ just flips the 'meaningful' low bits	$\sim$ flips ALL bits of the full width; mask with $\&$ afterward if only low bits matter
Right-shifting a negative signed int expecting zero-fill	Arithmetic shift sign-extends; use an unsigned type for a guaranteed logical shift
XOR-swapping a variable with itself	$a \oplus = a$ zeroes it out; use <code>std::swap</code> or add an $a \neq b$ guard
Checking every bit position to count set bits	Use $n \& (n-1)$ in a loop, or <code>__builtin_popcount</code> , to loop only once per SET bit
Using bitmask DP with n too large	2^n states must fit in time/memory — typically $n \leq 20$ is the practical ceiling
Forgetting the empty submask when iterating submasks with $(sub-1) \& mask$;	That loop stops at $sub=0$ without processing it — handle $mask=0$'s case separately if needed

Complexity Table

Operation	Complexity
Any single bitwise operator (&, , ^, ~, <<, >>)	$O(1)$
Counting set bits via $n \& (n-1)$ loop	$O(\text{number of set bits})$
Enumerating all subsets via bitmask	$O(2^n)$ subsets, $O(n)$ work each = $O(n \times 2^n)$
Enumerating all (mask, submask) pairs	$O(3^n)$ total
Bitmask DP (TSP-style, $O(n)$ transition)	$O(2^n \times n^2)$

Interview-Style Questions

- Why does two's complement make addition and subtraction work identically for positive and negative numbers?
- Explain why $n \& (n-1)$ removes exactly the lowest set bit, using a concrete binary example.
- Why does XORing an entire array find the single element that appears an odd number of times while all others appear an even number of times?
- Why is bitmask DP only practical for small n , and what specifically grows too large as n increases?
- Why does iterating all (mask, submask) pairs across every mask cost $O(3^n)$ rather than $O(4^n)$?

Cheat Sheet — Part 19

```

// Core bit tricks
bool isOdd    = n & 1;
bool kthBit   = (n >> k) & 1;
n = n | (1 << k);           // set bit k
n = n & ~(1 << k);          // clear bit k
n = n ^ (1 << k);           // toggle bit k
n = n & (n - 1);            // remove lowest set bit
int lowest = n & (-n);       // isolate lowest set bit
bool isPow2 = n > 0 && (n & (n - 1)) == 0;
int bits = __builtin_popcount(n); // count set bits
(unsigned)

// XOR tricks
int unique = 0;
for (int x : a) unique ^= x; // find single non-paired element

// Subset enumeration
for (int mask = 0; mask < (1 << n); mask++) {
    for (int i = 0; i < n; i++)
        if (mask & (1 << i)) { /* i is in this subset */ }
}

// Submask enumeration (submasks of a fixed mask)
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) { /* ...
*/ }

```

Where to go from here

With this gap filled, the roadmap's Phase 16 is now covered. If you'd like the remaining un-built Parts renumbered to match the original Part XIX onward (Advanced Algorithms next), just say so — otherwise, continuing with "Part 20" will pick up Advanced Algorithms (divide and conquer, meet in the middle, coordinate compression, sweep line, small-to-large) under the next available number.